AI Supply Chain Guardian

Clone-to-run developer guide

19 October 2026 | Local development | Linux / Ubuntu Bash

Run the backend first. Then connect the web app, CLI or VS Code extension to that same backend. You do not need three separate backends.

This guide uses the native Python setup as the main route and Docker Compose as an alternative. Commands are run from the repository root unless a step says otherwise. Keep the backend terminal open while using any client.

What starts where

- Backend: seven detector services plus the Trust Engine API. Default API: http://127.0.0.1:18108.
- Web: React/Vite interface on http://127.0.0.1:3000. It forwards /api requests to the backend.
- CLI: scripts/guardian_cli.py submits files to the running API.
- VS Code: the extension submits the active editor buffer to the running API.
- Copilot hook: optional, separately configured checks before supported file tools write.

Read this first

Install Git, Python 3.12 with venv support, Node.js 22.12 or newer, npm, and VS Code if you want editor integration. A GPU is not required. Allow several GB of free disk space and adequate RAM for CPU model inference. Downloads require internet access; registry/advisory checks can also require network access during scans.

Native commands below are for Linux. Windows users should use an Ubuntu WSL2 terminal for the backend/CLI and install the extension in the VS Code environment where they will use it. Native Windows and macOS execution were not validated in this audit.

The local judge route uses downloaded Qwen weights inside the service; it does not require Ollama or a provider API key. The default launcher otherwise selects Ollama, so keep the explicit --judge-provider local option shown here.

1. Clone and prepare the runtime

Step 1 - Get the repository

```
git clone https://github.com/Cipher-Sheild/AI_Supply_Chain_Guardian-v2.git
cd AI_Supply_Chain_Guardian-v2
```

Step 2 - Check prerequisites

```
python3.12 --version
node --version
npm --version
git --version
```

Python should report 3.12.x and Node should meet the 22.12+ requirement. If Python reports that ensurepip or venv is unavailable on Ubuntu, install the matching python3.12-venv package using your system package manager, then retry.

Step 3 - Install dependencies and pinned model data

```
python3.12 scripts/setup_local.py
```

The script creates .data/venv, installs the Python runtime and training dependencies, downloads the pinned MiniLM embedding and Qwen judge snapshots, installs both npm projects, and builds the VS Code extension. First setup can take time; wait for the final Runtime ready message. Do not interrupt a model download and assume setup completed.

Step 4 - Create a private client key

```
mkdir -p "$HOME/.config/guardian"
(umask 077; printf 'GUARDIAN_API_KEY=%s\n' \
  "$(openssl rand -hex 32)" > "$HOME/.config/guardian/client.env")
```

This creates a new credential. Run it once; re-running changes the key. Keep the file outside Git. The examples below use this same key for backend, web, CLI and editor access. OpenSSL must be available for this command.

Step 5 - Check the environment

```
.data/venv/bin/python -m pip check
```

Expected result: No broken requirements found. Downloaded weights, local data and node_modules are ignored by Git. You do not need to activate the virtual environment because the commands name its Python executable directly.

You do not need Hadolint, Trivy, Semgrep or ScanCode for the default Guardian-native scanner route. Optional external compatibility engines require their own tools and separate validation.

2. Start the backend first

Terminal A - Run all backend services

From the repository root, load the key you generated and start the backend:

```
set -a
. "$HOME/.config/guardian/client.env"
set +a
.data/venv/bin/python scripts/run_local.py \
  --python "$PWD/.data/venv/bin/python" \
  --judge-provider local
```

Leave this terminal running. The launcher starts all eight Python services, binds them to loopback, and keeps their logs under .data/. It uses local SQLite storage automatically; you do not need to start PostgreSQL for this native route. Redis caching is optional and can fall back when Redis is unavailable.

The default backend ports are 18101 through 18108. The Trust Engine, which clients use, is port 18108. Do not point the clients at an individual detector port.

Terminal B - Check readiness

Open another terminal in the same repository:

```
curl -s http://127.0.0.1:18108/health
```

Wait for status ok and successful checks. During model startup, a 503/degraded response is possible. If a check stays false, inspect the matching .data/<service>.log file. The launcher printing a URL does not mean every model is ready yet.

Stop or restart

Press Ctrl-C in Terminal A to stop the services started by that launcher. Run the same launch command to restart. Local scan history remains in .data/scans.sqlite3. Do not delete that directory as a troubleshooting shortcut if you need its history.

These instructions use an authenticated loopback-only backend. Source only the trusted file you generated; never source an untrusted deployment file. The launcher creates a separate callback token automatically. Keep services bound to loopback.

3. Start and use the web app

Keep Terminal A running

The web app needs the backend from page 3. Starting the frontend alone does not start the scanners or the database.

Terminal B - Start the frontend

```
TRUST_ENGINE_URL=http://127.0.0.1:18108 \
GUARDIAN_DASHBOARD_CLIENT_CONFIG="$HOME/.config/guardian/client.env" \
  npm run dev --prefix frontend -- \
  --host 127.0.0.1 --port 3000 --strictPort
```

Open http://127.0.0.1:3000 in your browser. Keep this terminal open too. Use 127.0.0.1 consistently when following this guide.

Submit a first scan

- Confirm the Local deployment access is ready banner. The private Vite proxy supplies the client key without returning it to the browser. If using a manually authenticated frontend, import the client file through Deployment access instead.
- Enter a supported filename, for example app.py.
- Paste source text or choose the provided file-input option.
- Select Scan artifacts and wait until the result is Final.
- Inspect Accept / Review / Reject, findings, coverage and scoring explanation.
- Open scan history or download the audit PDF.

Try this deliberately risky Python snippet as scanner input; do not execute it:

```
import os
os.system(input())
```

A completed scan is not necessarily Accept. Review and Reject are valid outcomes. An incomplete check must not be treated as a clean result. Container scans also distinguish supplied configuration from resolved-image inspection; read the displayed scope.

Alternative: one terminal starts both backend and frontend

Stop any existing native stack and frontend first. In the backend terminal where the key is loaded, run:

```
GUARDIAN_DASHBOARD_CLIENT_CONFIG="$HOME/.config/guardian/client.env" \
.data/venv/bin/python scripts/run_local.py \
  --python "$PWD/.data/venv/bin/python" \
  --judge-provider local --dashboard
```

Do not start a second frontend on port 3000 when --dashboard already owns it.

4. Use the CLI

Keep the backend running

The CLI is a client. It does not launch the eight backend services. You do not need the frontend for CLI scans.

Create a small harmless input file

In another terminal at the repository root:

```
mkdir -p .data/examples
printf 'print("Hello Guardian")\n' > .data/examples/hello.py
```

Scan one file

```
.data/venv/bin/python scripts/guardian_cli.py \
  .data/examples/hello.py --url http://127.0.0.1:18108 \
  --deployment-config "$HOME/.config/guardian/client.env"
```

The command prints JSON and waits for a final decision, subject to its timeout. A harmless input may still receive Review if evidence or model/provider readiness is incomplete.

Scan a project directory

```
.data/venv/bin/python scripts/guardian_cli.py \
  /absolute/path/to/your/project --url http://127.0.0.1:18108 \
  --deployment-config "$HOME/.config/guardian/client.env"
```

Replace the example path with a real directory. The CLI discovers supported source and configuration files, skips common generated directories, and submits bounded batches. It does not execute the project. A directory scan does not establish a complete dependency graph or inspect every binary file.

Interpret the exit code

- 0: final Accept and score meets the requested threshold.
- 2: Review or score below the threshold.
- 3: Reject.
- 4: connection, invalid response, input or incomplete-result error.

```
echo $?
```

Run this immediately after the scan to see its exit status. The default threshold is 80. Use --help for batch-size, timeout, authentication and certificate options.

5. Connect VS Code

Step 1 - Keep the backend running

Use Terminal A from page 3. The web app is optional for editor scans. The extension does not start the backend automatically.

Step 2 - Build and install the extension

```
npm ci --prefix integrations/vscode
npm run package --prefix integrations/vscode
code --install-extension integrations/vscode/guardian.vsix
```

The package command also builds the extension. The VSIX is generated locally and is not stored as a source artifact in Git. If the code command is not on PATH, open VS Code's Extensions menu and use Install from VSIX, choosing this generated file.

Step 3 - Configure the backend URL

Open VS Code Settings, search for guardian.serverUrl, and set it to:

```
http://127.0.0.1:18108
```

Alternatively, place this setting in the target workspace's .vscode/settings.json, preserving any existing settings:

```
{
  "guardian.serverUrl": "http://127.0.0.1:18108",
  "guardian.scanOnSave": false
}
```

Step 4 - Scan an artifact

Open a supported source or configuration file. Open the Command Palette and run Guardian: Scan Active Artifact. The extension submits the active editor buffer, waits for a final result, and displays diagnostics. Edit the file and scan again to check the revised content.

Before scanning, run Guardian: Import Client Key from Deployment Configuration and select $HOME/.config/guardian/client.env created on page 2. You can also use Guardian: Set API Key. Import only a trusted local configuration file; the extension stores the client key using VS Code SecretStorage.

Scan-on-save runs after a save. To check proposed Copilot writes before they happen, configure the separate hook on page 7.

6. Authentication and optional Copilot hook

Reuse the same deployment credential

The native route already created $HOME/.config/guardian/client.env on page 2. All clients must use its key. The launcher generates a distinct callback token; never substitute a provider key or callback token for the client API key.

- CLI: pass --deployment-config with the private client-file path.
- VS Code: import the private client file with the Guardian command.
- Web: use the private local proxy setting from page 4 or import the file through Deployment access.

Do not regenerate the file while a backend is using the old key. To rotate, stop the backend, generate and load the new key, restart, then update all clients. Never commit private configuration or paste credentials into source files.

Optional: Copilot pre-write checks

With an authenticated, healthy backend running, prepare a new separate workspace:

```
.data/venv/bin/python scripts/prepare_video_demo.py \
  --workspace "$HOME/GuardianSandbox" \
  --url http://127.0.0.1:18108 \
  --deployment-config "$HOME/.config/guardian/client.env"
code "$HOME/GuardianSandbox"
```

Trust that workspace, set guardian.serverUrl to the same API, enable chat.useHooks, and verify discovery with Chat: Configure Hooks. Use a Local Copilot Agent session and inspect Output > GitHub Copilot Chat Hooks for real executions. Do not confuse Local Agent with unrelated cloud sessions.

Supported file proposals require final Accept. Review, Reject and inspection failure deny the supported write. Manual edits, inline Tab completion, arbitrary tools and other harnesses are outside this boundary. Keep normal tool approvals enabled. The optional live comparison view needs separately configured comparison logging; ordinary hook setup does not create paired ML runs.

7. Alternative: Docker Compose

Use this instead of the native backend

Install Docker Engine/Desktop with Compose v2 and ensure docker info succeeds. Stop any process already using ports 8000 or 3000. Run commands from the checkout root.

Generate the private deployment configuration

```
python3.12 scripts/configure_deployment.py \
  --output .data/deployment/config.env
```

The generator refuses to overwrite existing credentials. It creates separate client, callback and database credentials. Do not commit the generated file.

Configure the judge before starting

Compose defaults to an external Ollama service and does not install it. Run Ollama separately, obtain the model you select, and set OLLAMA_MODEL and OLLAMA_HOST in the private config. Containers must be able to reach that host; host.docker.internal is mapped through host-gateway on Linux. An Ollama server bound only to host loopback may not be reachable from containers. Configure access for the Docker network without exposing it publicly.

Alternatively use a documented supported hosted provider with its private key and model settings. See docs/RUNTIME_CONFIGURATION.md. Unavailable judge or registry services can cause degraded health or Review.

Build, start and check

```
docker compose --env-file .data/deployment/config.env up --build -d
docker compose --env-file .data/deployment/config.env ps
curl -s http://127.0.0.1:8000/health
```

Compose starts PostgreSQL, Redis, migrations, all eight services and the web app. Initial builds/model loading can take time. Use http://127.0.0.1:3000 for the web and port 8000 for CLI/VS Code. Authenticate clients using the generated deployment config; callbacks use their separate token. If opening /docs, supply the API credential through an appropriate API client; browser navigation alone does not add a Bearer header.

Logs and shutdown

```
docker compose --env-file .data/deployment/config.env logs --tail=100 trust_engine
docker compose --env-file .data/deployment/config.env down
```

Volumes preserve history. Do not add -v unless you intend to delete stored data. This guide distinguishes native HTTP checks from a fresh complete Compose deployment; consult the audit record for what was actually verified.

8. Troubleshooting and checks

Connection refused

Start the backend and verify /health. Native uses 18108; Compose uses 8000. Match the URL in the CLI, VS Code and frontend. The web page opening successfully does not prove backend readiness.

Port already in use

Stop the process that owns the port or choose another range. For example, --port-base 19101 gives API 19108. Update every client and TRUST_ENGINE_URL accordingly. Change the frontend port separately if 3000 is occupied.

401 / 403 responses

A 401 means the configured client credential is missing or incorrect. Import the deployment client key, not a provider key or callback token. A 403 in keyless local mode can indicate an unexpected browser Host/Origin; use the documented loopback URLs and avoid foreign-origin requests.

Degraded health or repeated Review

Inspect .data/<service>.log or Compose logs. Verify completed model downloads, the judge configuration and network access to registries/advisory feeds. Missing evidence is not a clean verdict. A service being healthy also does not establish security accuracy.

Repeat the core development checks

```
.data/venv/bin/python scripts/verify_test_suites.py \
  --output .data/latest-test-results.json
.data/venv/bin/python scripts/sync_contracts.py --check
.data/venv/bin/python scripts/sync_runtime.py --check
npm run build --prefix frontend
npm run build --prefix integrations/vscode
```

The PostgreSQL test requires a separate migrated scratch database and TEST_DATABASE_URL. Never point tests at demonstration history. Browser tests require Playwright Chromium and the configured authenticated backend; VS Code tests require an editor test runtime. These are separate from the isolated service suites.

Repository map

- services/: independently deployable scanner services and Trust Engine.
- frontend/: web client and development proxy.
- integrations/vscode/: editor extension and pre-write documentation.
- runtime/ and contracts/: canonical shared code and contracts; use sync checks after editing.
- scripts/: setup, launch, clients, verification and model tooling.
- evaluation/: versioned datasets, labels, provenance, model evidence and limitations.
- docs/: architecture, reports, setup guide and audit results.
- .data/: ignored private runtime, logs and local verification outputs.

Weighted enforcement is the default; ML remains an evaluation candidate. A clean scan is not proof that the whole application is secure.